EXTENDER: A Multi-Agent System for Code Extensibility Review

Anonymous Author(s)

Abstract

Software systems must evolve as requirements and environments change, making extensibility a central quality goal. The SOLID principles, *i.e.*, five object-oriented design principles, are widely endorsed in software engineering practice to evaluate software quality. However, automatically detecting violations of SOLID principles remains difficult since many violations are latent design flaws that only surface when the code must be extended. In this paper, we propose to check the violations of SOLID principles by explicitly extending and checking the extensibility of existing code. Based on this idea, we design EXTENDER, a multi-agent system that detects SOLID violations by generating principle-specific extension requirements, asking AI agents to implement it, and checking whether the resulting diffs expose design defects. Experimental results show that EXTENDER achieves **87.8%** detection accuracy with a Qwen3.5 9B model, which is 32.1% higher than the state-of-the-art LLM-based approaches.

CCS Concepts

• **Software and its engineering** → **Software maintenance tools**;
• **Computing methodologies** → **Multi-agent planning**.

Keywords

Multi-Agent Systems, SOLID Principles, Software Extensibility

1 Introduction

Software extensibility is a central quality attribute since software usually evolves with requirements, users, and development environments. The SOLID principles give developers widely used guidance for preserving extensibility. They constrain coupling, responsibility boundaries, interfaces, and dependencies [2, 11, 12, 28].

Automatically detecting SOLID violations is thus an important research topic that has gained wide attention [3, 13, 15, 17, 18, 24]. Specifically, existing approaches fall into two categories, *i.e.*, code-analysis-based approaches and LLM-based approaches. The former apply static analysis to extract structural information and code metrics, and detect violations with predefined rules or thresholds over these syntactic proxies [3, 15, 17, 21, 24]. The latter instead rely on the semantic code understanding and reasoning capability of large language models (LLMs) to judge whether the code conforms to each principle [13, 18].

However, existing approaches still suffer from poor accuracy in detecting SOLID violations [13, 18]. We argue that the root cause lies in the fact that all existing approaches detect SOLID principle violations purely based on the current state of the code. In contrast, violations of SOLID principles depend on the cost of a future change. A design can look acceptable before it is challenged, and its defect becomes visible only when a new variant, client, or dependency is introduced. Since existing approaches reason only over the current version of the code, they cannot truly measure the cost that is paid only when the code is extended.

In this paper, we propose EXTENDER, a multi-agent system that detects SOLID violations by explicitly extending the code under review. Our key idea is to organize SOLID violation detection as a multi-agent extension experiment. Specifically, EXTENDER consists of four types of agents. A *manager* first creates five constrained code extension requests, one for each SOLID principle. Five parallel *engineers* then implement the requests on isolated copies of the input code. For each extended code, an *analyst* checks whether the difference between each extended version and the original code matches a principle-specific violation pattern. Finally, a *ranker* aggregates the analysis results and returns the most likely violations together with all supporting artifacts.

We evaluate EXTENDER with Qwen3.5 9B on 240 examples and compare it with two LLM-based baselines [18], *i.e.*, a single-agent baseline and a two-agent baseline under a detector-evaluator paradigm [14]. EXTENDER reaches 87.9% top-2 accuracy, which is 17.8 and 32.1 points higher than the single-agent and two-agent baselines. These results validate that explicitly extending the code exposes latent design flaws that facilitate SOLID violation detection.

This paper makes three contributions:

- We are the first to propose adopting LLM-based code extension to expose latent design flaws in code for detecting SOLID principle violations.
- We implement and open-source EXTENDER, a multi-agent system for code extensibility review[1].
- We evaluate EXTENDER on real-world SOLID violation detection datasets and show that EXTENDER can effectively improve detection accuracy by up to 32.1% compared with the state-of-the-art one-agent and two-agent baselines.

2 Background & Related Work

This section defines the design principles studied in this work and distinguishes traditional, LLM-based, and agentic approaches to code-quality evaluation.

2.1 SOLID Principles

SOLID principles are widely adopted in industry to improve code modularity and maintainability [2, 25, 28]. It comprises five object-oriented design principles [11, 12]. Specifically, the **Single Responsibility Principle (SRP)** limits a module to only one responsibility. The **Open/Closed Principle (OCP)** favors adding new behavior through extension instead of modifying stable implementation logic that is already tested. The **Liskov Substitution Principle (LSP)** requires subtypes to preserve the behavior of their base types, which ensures the correctness of newly implemented subtypes in existing systems. The **Interface Segregation Principle (ISP)** prevents clients from depending on operations they do not use, which

AgenticDev 2026, Munich, Germany
2026.

[1] We will make our code repository publicly available after the paper is accepted.

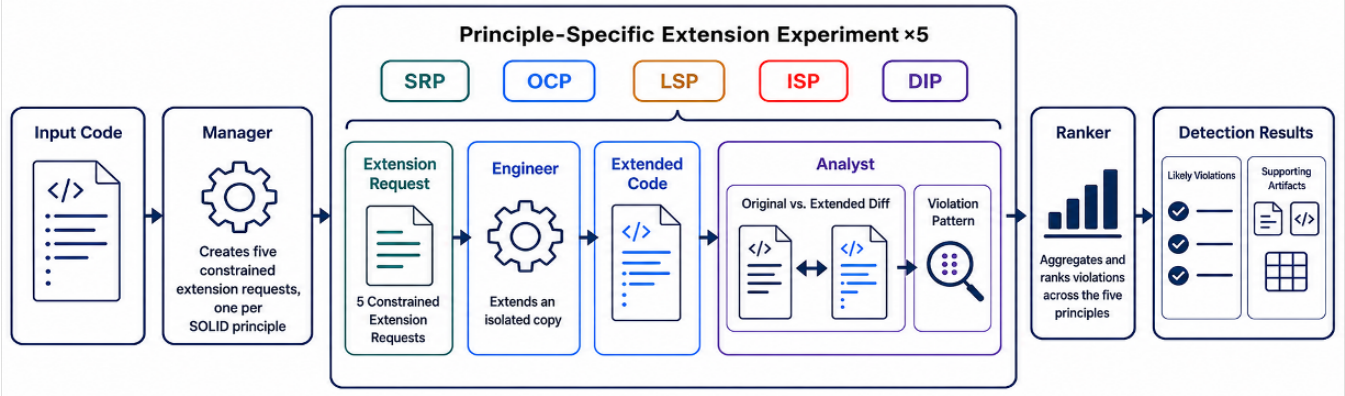

Figure 1: The Extender multi-agent workflow.

reduces the overhead of implementing dumb interfaces when extending the code. Finally, the **Dependency Inversion Principle (DIP)** makes high-level policy depend on abstractions rather than concrete implementations to simplify code extension.

These principles protect extensibility by reducing the impact of potential bugs induced by code changes. A violation is therefore not only a property of the current source structure. It often becomes observable when a new variant, subtype, client, or dependency forces unexpected edits.

2.2 Traditional Code Quality Evaluation

Traditional code-quality tools use static rules, source metrics, and structural proxies to identify code smells and design problems [17, 24]. For SOLID, existing techniques check class diagrams, coupling relationships, interface structure, and explicit dispatch logic [3, 15, 21]. These analyses are deterministic, repeatable, and inexpensive enough for continuous integration. Refactoring studies also assess SOLID indirectly by measuring a system before and after design improvements [2, 28].

However, existing static indicators work best when a violation has a clear structural signature. They cannot reliably recover architectural intent or the cost of a change that has not yet been attempted. Traditional evaluation thus describes the current program well, but it does not directly test how the program responds to a targeted extension.

2.3 LLM-Based SOLID Violation Detection

LLMs extend code analysis beyond fixed syntactic rules by using learned representations of programs and natural-language design knowledge [4–6, 22]. Recent work applies this capability to design-pattern recognition and software-quality assessment [13, 23]. For SOLID specifically, Martins et al. use an LLM to enforce principles during code review [13]. Pehlivan et al. systematically evaluate prompts and code models on a multilingual benchmark covering all five principles [18].

While existing LLM-based approaches improve semantic coverage, they still ask a model to infer a future design problem from the current code snapshot. Their accuracy consequently varies across principles, especially when a violation becomes visible only after the code is extended. Compared with existing LLM-based approaches, Extender changes the evidence available to the model by first generating a targeted extension and then analyzing the resulting difference in code.

2.4 Agentic Approaches to Code Quality

Agentic systems divide a complex task among roles for planning, acting, reflection, and coordination [26, 29]. In code-quality evaluation, prior tools combine static rules, machine learning, IDE assistance, or LLM feedback to detect and sometimes refactor test smells [1, 7, 10, 19, 27]. Melo et al. show that a detector-evaluator workflow can identify and refactor test smells with small language models [14].

Extender also assigns specialized agent roles, but organizes them for a different purpose. Existing agentic tools divide the detection task itself among roles, *e.g.*, a detector and an evaluator that reason over the program as given [14]. The agents in Extender instead cooperate to conduct a controlled extension experiment that produces the diff evidence discussed above, and each finding remains traceable to the extension request and the modified lines that produced it.

3 The Extender Multi-Agent System

3.1 System Overview

In this paper, we propose Extender, a multi-agent system that detects violations in SOLID principles by explicitly extending the code under review. As shown in Figure 1, Extender has four agents in a workflow. On receiving input code, it invokes a *manager* to create five constrained code extension requirements, one for each SOLID principle. Five parallel *engineers* then extend isolated code copies for each requirement. Given the extended code by each engineer, an *analyst* compares the extension with the original code and checks whether the code difference matches a principle-specific violation pattern. Finally, a *ranker* aggregates the analysis results and returns the most likely violations together with all supporting artifacts. Such a design enables us to adopt the difference between the original code and extended code to detect code extensibility issues, *i.e.*, violations of SOLID principles, which can expose more latent design errors in the current code base.

Table 1: Code extension requirements and violation patterns for each SOLID principle.

Principles	High-Level Requirement Generation Instructions	Principle Violation Patterns
SRP	Change one architectural layer.	The diff crosses independent layers, such as controller, service, repository, or view.
OCP	Add a type, variant, or case.	The extension edits an existing `if-else` or `switch` branch instead of adding an extension.
LSP	Exercise parent and subtype behavior polymorphically.	A subtype throws or breaks the contract expected from the parent.
ISP	Implement only a needed subset of an interface.	The implementation is forced to add unused methods or `NotImplementedError` stubs.
DIP	Replace one concrete dependency.	The high-level module must edit several concrete references because no useful abstraction is present.

In the rest of this section, we first introduce the detailed design of each agent. We then demonstrate a working example of SOLID violation detection with our system. Finally, we will briefly discuss our implementation.

3.2 Agent Design

In this subsection, we introduce the detailed design of the four agents of Extender respectively.

3.2.1 Manager. The *manager* is responsible for creating a concrete extension requirement for each SOLID principle. Since the input code varies from case to case, we design a high-level requirement generation instruction for each principle to expose the specific code design flaws. The second column of Table 1 shows our high-level instructions. The *manager* then instantiates it into a code-specific requirement.

Specifically, for **SRP**, we instruct the *manager* to design an extension requirement that modifies only one architectural layer. Code with potential **SRP** violations can then involve changes in other layers. For **OCP**, the high-level instruction is to add a new type, variant, or case to the current implementation. Code with **OCP** violations may potentially modify existing code rather than adding new code. For **LSP**, the instruction is to exercise the parent type and its subtypes polymorphically. Code with **LSP** violations can expose bugs when using subtypes in applications designed for parent types. For **ISP**, the instruction is to implement only a needed subset of an existing interface. Code that violates **ISP** may then be forced to carry methods it does not need. Finally, for **DIP**, the instruction is to replace one concrete dependency with an alternative. A violation can be detected by checking whether the high-level module is edited due to missing abstractions.

3.2.2 Engineer. Given the requirement generated by the *manager*, a dedicated *engineer* implements it and produces a revised program. We isolate *engineers* for each SOLID principle to reduce the context length of each agent since agents with long contexts usually result in suboptimal results [9].

3.2.3 Analyst. The *analyst* is responsible for detecting SOLID violation for each principle based on the code extension requirement, the original code, and the revised code. For each principle, a verdict that indicates whether the principle is violated and a reason is generated. Specifically, the *analyst* first computes a unified diff between the extended program and the original code with a deterministic textual diff tool. It then checks whether the diff matches the violation pattern configured for the target principle.

The violation patterns are summarized in the third column of Table 1. For **SRP**, the *analyst* confirms a violation whenever the change requested within a single architectural layer is forced to spread into other layers, such as controller, service, repository, or view. For **OCP**, the *analyst* reports a violation whenever the new variant is wired in by editing an existing `if-else` or `switch` branch instead of adding new code along an extension point. For **LSP**, the *analyst* flags a violation whenever a subtype throws an exception or otherwise breaks the contract expected from its parent under polymorphic calls. For **ISP**, the *analyst* confirms a violation whenever the new implementation is forced to carry unused methods or `NotImplementedError` stubs. Finally, for **DIP**, the *analyst* reports a violation whenever replacing one concrete dependency forces edits in the high-level module because no useful abstraction is present.

3.2.4 Ranker. Finally, the *ranker* aggregates the five verdicts into the detection result in a single call. Given the original program and the verdicts from all principles, the *ranker* first discards the principles with negative decisions or unsatisfied structural prerequisites. It then ranks the remaining candidate violations by how evident they are, considering two criteria: (1) the clarity of the evidence, and (2) the strength of the explanation. The *ranker* reports the top-k candidates (k=2 by default) as the most likely violations. Each candidate is accompanied by its supporting artifacts, including the extension requirement, the diff, and the *analyst*'s explanation. Developers can trace every reported violation back to the request and the modified lines that produced it.

3.3 A Working Example

Listing 1 shows an example from an actual run of Extender on the benchmark code `OCP_25`. The original code calculates the area of various shapes, *i.e.*, the `AreaCalculator` class dispatches on concrete shape types with a `when` expression.

The trace shows how the four agents cooperate on a concrete input. Given the high-level OCP instruction, the *manager* instantiates it into a code-specific requirement, *i.e.*, adding support for a `Triangle` shape to the area calculation system. The *engineer* adds a `Triangle` class and edits the existing `when` branch inside

Listing 1: An Extender execution trace for benchmark example OCP_25.

```
Manager -> requirement r_OCP:
  "Add support for a Triangle shape to the area
   calculation system."

Engineer -> diff d_OCP (abridged):
+ class Triangle(private val base: Double,
+   private val height: Double) : Shape("Triangle") {...}
  class AreaCalculator {
    fun calculateArea(shape: Shape): Double {
      return when (shape) {
        is Rectangle -> shape.getWidth() * shape.getHeight()
        is Circle -> Math.PI * shape.getRadius() * ...
+       is Triangle -> 0.5 * shape.getBase() * ...
        else -> 0.0
      }
    }
  }

Analyst -> verdict v_OCP: violation detected
  "The AreaCalculator class violates the Open/Closed
   Principle because its calculateArea() method contains
   a conditional block (when expression) that directly
   handles specific shape types. ... This structure
   requires modifying the existing decision logic ...
   rather than extending the system through new
   polymorphic types or dedicated handlers."

Ranker -> top-2: [OCP, DIP]
  (ground truth: OCP, reported at rank 1)
```

`AreaCalculator.calculateArea`, which violates OCP. The *analyst* accordingly returns a positive verdict and explains that accommodating new shapes requires modifying the existing decision logic instead of extending the system through new polymorphic types. The *ranker* finally arbitrates among the channels. In this example, the SRP, ISP, and DIP channels also return positive verdicts, and the *ranker* places OCP first and DIP second because their explanations identify the most clearly located evidence.

3.4 Implementation

Extender is implemented as a LangGraph state machine [8] that accesses either a local Ollama server [16] or any LangChain-compatible provider. The evaluated configuration runs Qwen3.5 9B locally [20], and unified diffs are computed deterministically with Python's `difflib`.

In our implementation, we use five principle channels, each recording their own requirement, generated extension, unified diff, and *analyst* verdict in the workflow state. The final *ranker* pass reads only these recorded artifacts. Because the channels are mutually independent, *engineer* execution can be parallelized in deployment, and a failed or malformed extension is retained as an inconclusive artifact without blocking the remaining channels. The full system and analysis scripts are packaged for release. The public link is withheld from this anonymous version.

4 Evaluation

4.1 Experimental Setup

We evaluate Extender on the SOLID benchmark of Pehlivan et al. [18]. It contains 240 labeled examples, 48 for each principle. The 240 examples span four programming languages, *i.e.*, Java, Kotlin, Python, and C#, and three difficulty levels, *i.e.*, easy, moderate, and hard. There are 60 examples for each programming language and 80 examples for each difficulty. Each example has one ground-truth violation label.

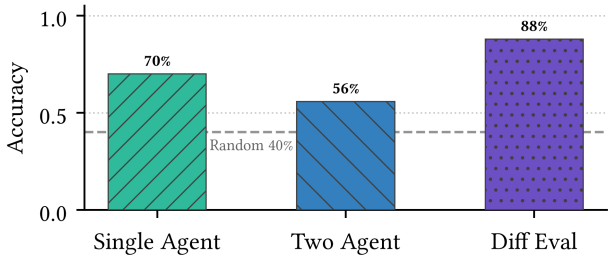

Figure 2: Overall accuracy on 240 examples.

We compare Extender with three workflows. A one-agent baseline directly prompts an LLM to identify a violation, which is proposed by Pehlivan *et al.* [18]. A two-agent baseline uses a detector agent and an evaluator agent for up to three feedback iterations. The two-agent baseline is originally proposed by Melo *et al.* [14] for code smell detection. We revise their prompt to make it work for SOLID violation detection. All three workflows use the same Qwen3.5 9B model deployed on a desktop with an NVIDIA RTX4060 desktop GPU.

For each example, all workflows receive the same source code and return a ranked list of SOLID labels. We configure the systems to return two labels because code can exhibit related design symptoms, while the benchmark supplies one primary label. We preserve the original runs and use fixed configurations for the reported comparison. The one-agent and two-agent baselines have access to the same principle definitions as Extender. The difference is that they reason directly over the source and their conversational context rather than a set of generated extension diffs.

4.2 Metrics

Throughout this paper, accuracy refers to top-2 accuracy, *i.e.*, a prediction is correct when the ground-truth violation appears in the first two ranked results. We therefore define accuracy as:

$$\text{Accuracy} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}[y_i \in \{\hat{y}_i^{(1)}, \hat{y}_i^{(2)}\}]. \quad (1)$$

We also report Mean Reciprocal Rank (MRR@2) and Macro-F1. MRR@2 rewards systems that place the correct result first. For Macro-F1, we adopt a resolved prediction to comply with the top-2 accuracy. Specifically, when the ground-truth label appears in the returned pair, that hit is resolved to the ground-truth class. Otherwise, the first valid prediction is used.

4.3 Main Results

Figure 2 shows that Extender improves accuracy by 17.9 points over the one-agent baseline and by 32.1 points over the two-agent baseline. The gain in MRR@2 (Table 2) shows that the multi-agent system also ranks correct results higher. The Macro-F1 score confirms that its improvement is not limited to one principle.

Table 2: Performance of local workflows and cloud inference.

System	Accuracy	MRR@2	Macro-F1
One Agent (Local)	70.00%	0.638	0.699
Two Agent (Local)	55.83%	0.490	0.576
EXTENDER (Local)	**87.92%**	**0.792**	**0.888**
One Agent (Cloud)	95.83%	0.858	0.960

Figure 3: Accuracy by different principles.

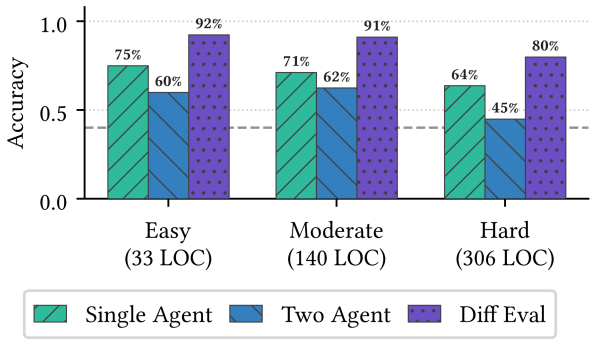

Figure 4: Accuracy by difficulty.

Analysis on each principle. The EXTENDER leads on SRP, LSP, ISP, and DIP (Figure 3). Compared with the two baselines, EXTENDER improves most on DIP. This is because tight coupling becomes clear when an *engineer* attempts to replace a concrete dependency. However, EXTENDER has suboptimal accuracy on OCP. This is because a direct prompt can often recognize a visible dispatch branch without an extension experiment.

4.4 Difficulty and Cloud Reference

Analysis on different difficulty levels. EXTENDER remains at 80% on Hard examples (Figure 4). This is because the *analyst* receives a focused diff rather than all generated context. This helps the multi-agent system concentrate on modifications that carry review evidence. It also reduces the impact of long source inputs.

Analysis on more powerful LLMs. We also compare EXTENDER with the single-agent baseline on more powerful LLMs on the cloud. Specifically, we use Qwen3.5 Plus with 397 billion parameters, which is over 44× larger than the Qwen3.5 9B model used by EXTENDER. As shown in Table 2, EXTENDER can maintain 93% of accuracy despite the huge gap in model capabilities. This result shows that EXTENDER is a strong lever for independent developers to deploy a smaller model on their own devices to conduct local code review. The development cost can be significantly reduced compared with invoking cloud APIs.

5 Discussion and Future Work

Generality. We argue that EXTENDER is a general paradigm for code extensibility review rather than a tool for detecting SOLID principle violations. The key idea is to adopt the extend-then-detect paradigm to detect code extensibility issues. We instantiate the paradigm on SOLID detection by designing EXTENDER. Since individual principles in SOLID enter the workflow only as a pair of natural-language artifacts, any design rule whose defect manifests as resistance to change can be plugged in by defining a new pair.

However, one limitation is that the paradigm is best suited to scenarios where an extension is genuinely needed to expose a latent design flaw. This is confirmed by the fact that EXTENDER underperforms the direct prompt on OCP, whose violations often carry a visible syntactic symptom such as an explicit dispatch branch. In the future, we plan to add a lightweight *router* in front of the workflow that predicts whether a violation is already detectable from the current state of the code.

Explainability. EXTENDER is designed for strong explainability. The *manager* request, the *engineer* extension, the executed diff, the *analyst* verdict, and the final rationale from the *ranker* are all recorded. Developers can trace a finding to the exact changed lines and accept or reject it on evidence. The same trace also separates planning, generation, and analysis errors, which turns a wrong finding into a diagnosable event instead of an opaque failure.

In the future, we plan to close the loop with reviewer feedback to evolve the performance of EXTENDER. Specifically, when a developer rejects a finding, the recorded trace identifies the responsible stage, so the corresponding instruction or violation pattern can be refined automatically rather than by global prompt tuning.

Repository-scale evaluation. The curated, self-contained benchmark enables controlled workflow comparisons but limits external validity because repository-scale violations may span files and principles [18]. Future work will evaluate EXTENDER on repository-level datasets and change histories.

6 Conclusion

We identify static-state reasoning as a key limitation of SOLID violation detection and propose an extend-then-detect paradigm that exposes latent design flaws through controlled code extensions. EXTENDER operationalizes this idea with four agent roles and diff-based evidence. On 240 examples, it outperforms the one-agent and two-agent baselines by 17.9 and 32.1 points, respectively.

References

[1] Wajdi Aljedaani et al. 2021. Test Smell Detection Tools: A Systematic Mapping Study. In *Proceedings of EASE*. 170–180.

[2] Apostolos Ampatzoglou et al. 2019. Applying the Single Responsibility Principle in Industry: Modularity Benefits and Trade-offs. In *Proceedings of EASE*. 347–352.

[3] Elena Chebanyuk and Konstantin Markov. 2016. An Approach to Class Diagrams Verification According to SOLID Design Principles. In *Proceedings of MODELSWARD*. 435–441.

[4] Mark Chen et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).

[5] Angela Fan et al. 2023. Large Language Models for Software Engineering: Survey and Open Problems. *arXiv preprint arXiv:2310.03533* (2023).

[6] Daya Guo et al. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming. *arXiv preprint arXiv:2401.14196* (2024).

[7] Stefano Lambiase et al. 2020. Just-in-Time Test Smell Detection and Refactoring: The DARTS Project. In *Proceedings of ICPC*. 441–445.

[8] LangChain Team. 2024. LangGraph: Building Stateful, Multi-Actor Applications with LLMs. https://github.com/langchain-ai/langgraph

[9] Mosh Levy, Alon Jacoby, and Yoav Goldberg. 2024. Same Task, More Tokens: The Impact of Input Length on the Reasoning Performance of Large Language Models. In *Proceedings of ACL*. 15339–15353.

[10] K. Lucas et al. 2024. Evaluating Large Language Models in Detecting Test Smells. In *Proceedings of SBES*. 672–678.

[11] Robert C. Martin. 2000. *Design Principles and Design Patterns.* Technical Report. Object Mentor.

[12] Robert C. Martin. 2017. *Clean Architecture: A Craftsman's Guide to Software Structure and Design.* Prentice Hall.

[13] G. F. Martins, E. C. M. Firmino, and V. P. De Mello. 2024. The Use of Large Language Model in Code Review Automation: An Examination of Enforcing SOLID Principles. In *Artificial Intelligence in HCI*. 86–97.

[14] Rian Melo et al. 2025. Agentic SLMs: Hunting Down Test Smells. arXiv:2504.07277 [cs.SE]

[15] Istika Oktafiani and Bayu Hendradjaya. 2018. Software Metrics Proposal for Conformity Checking of Class Diagram to SOLID Design Principles. In *Proceedings of ICoDSE*. 1–6.

[16] Ollama Team. 2024. Ollama: Run Large Language Models Locally. https://ollama.com

[17] Fabio Palomba, Gabriele Bavota, Massimiliano Di Penta, Fausto Ferrucci, Rocco Oliveto, and Andrea De Lucia. 2018. On the Diffuseness and the Impact on Maintainability of Code Smells. In *Proceedings of ICSE*. 1188–1198. doi:10.1145/3180155.3180220

[18] Fatih Pehlivan, Arcın Ülkü Ergüzen, Sahand Moslemi Yengejeh, Mayasah Lami, and Anil Koyuncu. 2025. Are We SOLID Yet? An Empirical Study on Prompting LLMs to Detect Design Principle Violations. In *Proceedings of ICSE*.

[19] Vincenzo Pontillo et al. 2024. Machine Learning-Based Test Smell Detection. *Empirical Software Engineering* 29, 2 (2024), 1–44.

[20] Qwen Team. 2026. Qwen3.5: Accelerating Productivity with Native Multimodal Agents. https://qwen.ai/blog?id=qwen3.5

[21] G. Roy, M. M., and B. A. Jacob. 2024. Automated Verification of Open/Closed Principle: A Code Analysis Approach. In *Proceedings of INCET*. 1–7.

[22] Baptiste Rozière et al. 2024. Code Llama: Open Foundation Models for Code. *arXiv preprint arXiv:2308.12950* (2024).

[23] Christian Schindler and Andreas Rausch. 2025. LLM-Based Design Pattern Detection. *arXiv preprint arXiv:2502.18458* (2025).

[24] Tushar Sharma and Diomidis Spinellis. 2018. A Survey on Software Smells. *Journal of Systems and Software* 138 (2018), 158–173. doi:10.1016/j.jss.2017.12.034

[25] Ozan Turan and Özlem Özgöbek Tanrıöver. 2018. An Experimental Evaluation of the Effect of SOLID Principles to Microsoft VS Code Metrics. *AJIT-e* 9, 34 (2018), 7–24.

[26] Lei Wang et al. 2024. A Survey on Large Language Model Based Autonomous Agents. *Frontiers of Computer Science* 18, 6 (2024). doi:10.1007/s11704-024-40231-1

[27] Tiantian Wang et al. 2021. PyNose: A Test Smell Detector for Python. In *Proceedings of ASE*. 593–605.

[28] Ivan Yanakiev, Bogdan-Mihai Lazar, and Andrea Capiluppi. 2025. Applying SOLID Principles for the Refactoring of Legacy Code: An Experience Report. *Journal of Systems and Software* 220 (2025), 112254.

[29] Shunyu Yao et al. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629* (2023).